

Optimization

*Lecturer: Jiaming Liang**September 22, 2026*

1 Gradient Descent

In the previous lectures, we introduced several loss functions for fitting machine learning models. This raises a natural question:

Once we have defined a loss function, how do we actually minimize it?

For some models, such as ordinary least squares, the optimal parameters can be written in closed form. For many other models, including logistic regression and neural networks, no convenient closed-form solution is available. Instead, we typically use an iterative optimization algorithm.

Suppose that our goal is to solve

$$\min_{\boldsymbol{\theta} \in \mathbb{R}^p} J(\boldsymbol{\theta}),$$

where $J(\boldsymbol{\theta})$ is the training objective and $\boldsymbol{\theta}$ contains the model parameters. Gradient descent is one of the most basic and important algorithms for solving this problem.

1.1 The Gradient as a Direction of Change

Recall that the gradient of J at $\boldsymbol{\theta}$ is

$$\nabla J(\boldsymbol{\theta}) = \begin{bmatrix} \frac{\partial J}{\partial \theta_1} \\ \vdots \\ \frac{\partial J}{\partial \theta_p} \end{bmatrix}.$$

The gradient points in the direction of the steepest local increase of the function. Consequently,

$$-\nabla J(\boldsymbol{\theta})$$

points in the direction of the steepest local decrease. To see this more explicitly, consider moving from $\boldsymbol{\theta}$ in a direction $\mathbf{d}$. For a small step $\eta > 0$, a first-order Taylor approximation gives

$$J(\boldsymbol{\theta} + \eta \mathbf{d}) \approx J(\boldsymbol{\theta}) + \eta \nabla J(\boldsymbol{\theta})^\top \mathbf{d}.$$

If we choose

$$\mathbf{d} = -\nabla J(\boldsymbol{\theta}),$$

then

$$J(\boldsymbol{\theta} - \eta \nabla J(\boldsymbol{\theta})) \approx J(\boldsymbol{\theta}) - \eta \|\nabla J(\boldsymbol{\theta})\|^2.$$

Thus, as long as the step size is sufficiently small, moving in the negative gradient direction decreases the objective.

1.2 The Gradient Descent Algorithm

Starting from an initial point $\boldsymbol{\theta}^{(0)}$, gradient descent repeatedly updates the parameters according to

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla J(\boldsymbol{\theta}^{(t)}),$$

where $\eta > 0$ is called the *step size* or *learning rate*. Each iteration consists of two basic operations:

1. Compute the gradient $\nabla J(\boldsymbol{\theta}^{(t)})$.
2. Move a distance proportional to the gradient in the opposite direction.

The algorithm stops after a prescribed number of iterations, or when the gradient becomes sufficiently small. A point satisfying

$$\nabla J(\boldsymbol{\theta}) = \mathbf{0}$$

is called a *stationary point*. Gradient descent attempts to move toward such points.

1.3 Learning Rate

The learning rate η determines how far gradient descent moves at each iteration. If η is very small, each iteration makes only a small amount of progress. The algorithm may converge, but it can require many iterations. If η is too large, the algorithm may overshoot the minimum. The objective may oscillate or even increase from one iteration to the next. The appropriate scale of the learning rate depends on how quickly the gradient of the objective can change.

A differentiable function J is called L -smooth if

$$\|\nabla J(\boldsymbol{\theta}) - \nabla J(\boldsymbol{\theta}')\| \leq L \|\boldsymbol{\theta} - \boldsymbol{\theta}'\|$$

for all $\boldsymbol{\theta}$ and $\boldsymbol{\theta}'$. Smoothness means that the gradient cannot change arbitrarily quickly. An L -smooth function satisfies

$$J(\boldsymbol{\theta}') \leq J(\boldsymbol{\theta}) + \nabla J(\boldsymbol{\theta})^\top (\boldsymbol{\theta}' - \boldsymbol{\theta}) + \frac{L}{2} \|\boldsymbol{\theta}' - \boldsymbol{\theta}\|^2.$$

For one gradient descent step, $\boldsymbol{\theta}' = \boldsymbol{\theta} - \eta \nabla J(\boldsymbol{\theta})$, we therefore obtain

$$J(\boldsymbol{\theta}') \leq J(\boldsymbol{\theta}) - \eta \|\nabla J(\boldsymbol{\theta})\|^2 + \frac{L\eta^2}{2} \|\nabla J(\boldsymbol{\theta})\|^2 = J(\boldsymbol{\theta}) - \eta \left(1 - \frac{L\eta}{2}\right) \|\nabla J(\boldsymbol{\theta})\|^2.$$

In particular, if $0 < \eta \leq \frac{1}{L}$, then

$$J(\boldsymbol{\theta}') \leq J(\boldsymbol{\theta}) - \frac{\eta}{2} \|\nabla J(\boldsymbol{\theta})\|^2.$$

Thus, with a sufficiently small learning rate, every nonstationary gradient descent step decreases the objective. This result also explains why choosing a learning rate that is too large can cause instability: the first-order approximation is only reliable over a sufficiently small region.

1.4 Convex and Nonconvex Objectives

The behavior of gradient descent depends strongly on the geometry of the objective. If J is convex, then every local minimum is also a global minimum. In particular, for a differentiable convex function,

$$\nabla J(\boldsymbol{\theta}^*) = \mathbf{0}$$

implies that $\boldsymbol{\theta}^*$ is a global minimizer. This is the situation for many classical machine learning models. For example, the least-squares objective and the logistic regression objective are convex functions of their parameters. For nonconvex objectives, the situation is more complicated. A stationary point may be a local minimum, a local maximum, or a saddle point. Moreover, different initializations may lead gradient descent to different stationary points. Neural networks typically lead to nonconvex optimization problems. Nevertheless, variants of gradient descent remain the standard tools for training them.

2 Stochastic Gradient Descent

In many machine learning problems, the objective function is an empirical average over the training data:

$$J(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n L_i(\boldsymbol{\theta}), \quad \nabla J(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \nabla L_i(\boldsymbol{\theta}),$$

where $L_i(\boldsymbol{\theta})$ denotes the loss associated with the i -th training example. Gradient descent therefore requires computing the gradient of every training example before making a single parameter update. When the dataset is large, this can be expensive. This motivates *stochastic gradient descent* (SGD).

2.1 Stochastic Gradients and SGD

Instead of computing the full gradient, suppose that we randomly select one training example i and use

$$\mathbf{g}(\boldsymbol{\theta}) = \nabla L_i(\boldsymbol{\theta})$$

as an approximation to the full gradient. If i is selected uniformly at random from $\{1, \dots, n\}$, then

$$\mathbb{E}[\mathbf{g}(\boldsymbol{\theta})] = \frac{1}{n} \sum_{i=1}^n \nabla L_i(\boldsymbol{\theta}) = \nabla J(\boldsymbol{\theta}).$$

Thus, the stochastic gradient is an *unbiased estimator* of the full gradient.

The stochastic gradient is generally noisy. Its variance can be measured by

$$\text{Var}(\mathbf{g}(\boldsymbol{\theta})) := \mathbb{E}[\|\mathbf{g}(\boldsymbol{\theta}) - \nabla J(\boldsymbol{\theta})\|^2].$$

A common assumption is that the variance is uniformly bounded:

$$\mathbb{E}[\|\mathbf{g}(\boldsymbol{\theta}) - \nabla J(\boldsymbol{\theta})\|^2] \leq \sigma^2 \quad \text{for all } \boldsymbol{\theta}.$$

Starting from an initial point $\boldsymbol{\theta}^{(0)}$, SGD performs the update

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta_t \nabla L_{i_t}(\boldsymbol{\theta}^{(t)}),$$

where i_t is a randomly selected training example and $\eta_t > 0$ is the learning rate.

Compared with gradient descent,

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta_t \nabla J(\boldsymbol{\theta}^{(t)}),$$

SGD replaces the exact gradient by a random, unbiased approximation. Each iteration is much cheaper, but the resulting trajectory is typically noisier.

2.2 Mini-Batch SGD

In practice, we usually use a small subset of the training data rather than a single example. Let

$$\mathcal{B}_t = \{i_1, \dots, i_B\}$$

be a mini-batch of size B , and define

$$\mathbf{g}_t = \frac{1}{B} \sum_{j=1}^B \nabla L_{i_j}(\boldsymbol{\theta}^{(t)}).$$

The mini-batch SGD update is

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta_t \mathbf{g}_t.$$

If the examples $i_1, \dots, i_B$ are sampled independently and uniformly, then the mini-batch gradient remains unbiased, i.e.,

$$\mathbb{E}[\mathbf{g}_t \mid \boldsymbol{\theta}^{(t)}] = \nabla J(\boldsymbol{\theta}^{(t)}).$$

Moreover, averaging multiple stochastic gradients reduces the variance. Let

$$\boldsymbol{\xi}_j = \nabla L_{i_j}(\boldsymbol{\theta}) - \nabla J(\boldsymbol{\theta}).$$

Then $\mathbb{E}[\boldsymbol{\xi}_j] = \mathbf{0}$, and under the uniform variance bound $\mathbb{E}\|\boldsymbol{\xi}_j\|^2 \leq \sigma^2$. Using independence,

$$\mathbb{E}[\|\mathbf{g}_t - \nabla J(\boldsymbol{\theta})\|^2] = \frac{1}{B^2} \mathbb{E}\left[\left\|\sum_{j=1}^B \boldsymbol{\xi}_j\right\|^2\right] = \frac{1}{B^2} \sum_{j=1}^B \mathbb{E}\|\boldsymbol{\xi}_j\|^2 \leq \frac{\sigma^2}{B}.$$

Thus, increasing the batch size reduces the variance of the gradient estimate by a factor of B . At the same time, larger batches require more computation per update. Mini-batching therefore provides a compromise between the cheap but noisy updates of single-sample SGD and the expensive exact updates of full gradient descent. The special cases are

$$B = 1 \quad (\text{single-sample SGD}), \quad 1 < B < n \quad (\text{mini-batch SGD}), \quad B = n \quad (\text{full gradient descent}).$$

2.3 Epochs and Shuffling

An *iteration* is one parameter update using one mini-batch, while an *epoch* is one complete pass through the training dataset.

For example, if the dataset contains $n = 60,000$ examples and the batch size is $B = 100$, then one epoch contains approximately

$$\frac{60,000}{100} = 600$$

iterations.

In practice, the training data are typically shuffled at the beginning of each epoch and then divided into mini-batches. Shuffling prevents the ordering of the data from introducing systematic bias into consecutive mini-batches. For example, if the data are sorted by class, then without shuffling several consecutive mini-batches may contain examples from only one class.

A typical training procedure is therefore:

1. shuffle the training data;
2. divide the data into mini-batches;
3. perform one SGD update for each mini-batch;
4. repeat for multiple epochs.